

Explication du codes Cplex et du code heuristique

Code de la solution Cplex :

Le code résout un problème de livraison de clients en utilisant un modèle de programmation linéaire avec la bibliothèque Cplex. Le problème consiste à trouver les tournées optimales pour un certain nombre de véhicules desservant plusieurs clients, tout en respectant des contraintes telles que la capacité des véhicules, les coûts de déplacement entre les clients, et d'autres restrictions.

Voici l'explication détaillée de chaque segment de code :

- `Cpx` : Le modèle crée à partir de « `cpx=Cplex()` ».
- `costs` : Une matrice de coûts qui représente le coût pour chaque véhicule de se déplacer entre deux villes.
- `D` : La liste des demandes des clients (quantité de produits à livrer ou récupérer).
- `truck` : Le nombre de véhicules disponibles.
- `Q` : La capacité maximale de chaque véhicule.
- `ntown` : Le nombre total de villes (clients + entrepôt).

Les variables de décision sont ajoutées au modèle. Elles sont divisées en trois types :

- `var_x` : Les variables binaires qui indiquent si un véhicule k voyage de la ville i à la ville j .
- `var_y` : Les variables binaires qui indiquent si un véhicule k est affecté à la ville i .
- `var_u` : Les variables continues qui représentent l'ordre dans lequel chaque ville est visitée par un véhicule.

Les contraintes sont ajoutées pour s'assurer que le modèle respecte toutes les règles du problème. Voici les principales contraintes :

- **Contrainte 1** : Chaque client doit être visité exactement une fois par un véhicule.
La boucle va jusqu'à `ntown-1` pour faire en sorte que l'entrepôt ne soit pas prise en compte vu qu'on y passe plusieurs fois.
- **Contrainte 2** : La somme des demandes des clients desservis par un véhicule ne doit pas dépasser la capacité du véhicule (Q).
La boucle va jusqu'à `ntown-1` puisque l'entrepôt n'est pas pris en compte dans une limite de capacité ou non.
- **Contrainte 3** : Chaque ville doit avoir un flux entrant et sortant équilibré (un véhicule entre et sort de chaque ville).
On fait en sorte de faire attention à ne pas avoir de flux entrant et sortant d'une même ville car on ne peut faire du surplace ainsi on précise bien dans nos boucles que i et j sont différents.
- **Contrainte 4** : Le dépôt doit être visité exactement une fois par chaque véhicule, à la fois comme point de départ et d'arrivée.
- **Contrainte 5** : La relation entre les variables x et y garantit que si un véhicule visite une ville, il doit quitter cette ville.
Ainsi nos boucles i et j pour `var_x` doivent bien aller chacune jusqu'à `ntown` et évidemment i et j sont différents (sinon cela pose problème comme dit précédemment).
- **Contrainte MTZ** : Les contraintes de Miller-Tucker-Zemlin pour éviter les sous-tournées dans la solution. Ces contraintes sont essentielles pour garantir que la solution finale ne contient pas de cycles non valides (comme j'ai pu avoir dans les différents codes que je vous avais montré avant celui-ci). En voici la formule où I représente le nombre de villes :

$$u_i - u_j + I * x_{ij} \leq I - 1 \quad \forall i, j \in I, j \neq 1, i \neq j$$
$$u_i \geq 0 \quad \forall i \in I$$

L'objectif du problème est de minimiser le coût total de la tournée. Le modèle définit la fonction objective en minimisant les coûts de déplacement entre les villes, en fonction des variables x.

Le modèle est ensuite résolu en utilisant la méthode « solve() » de Cplex.

Dans notre cas on obtient en retour de la part de la console :

- Le coût total de la solution optimale étant ici de 545€.
- Les var_X qui sont égales à 1, indiquant les trajets empruntés par chaque véhicule k que voici :

```
Optimal cost: 545.00 €  
x_0_7_1: 1.0  
x_1_3_1: 1.0  
x_2_7_2: 1.0  
x_3_0_1: 1.0  
x_4_7_0: 1.0  
x_5_4_0: 1.0  
x_6_2_2: 1.0  
x_7_1_1: 1.0  
x_7_5_0: 1.0  
x_7_6_2: 1.0
```

Code de la solution heuristique :

Le code présenté résout un problème de livraison de clients en utilisant une heuristique utilisant le principe du voisin le plus proche qui me paraissait être la méthode heuristique la plus appropriée à notre problème. Cette approche cherche à construire une tournée pour un véhicule en choisissant à chaque étape la ville la plus proche du véhicule tout en respectant la contrainte de capacité du véhicule.

L'algorithme commence par initialiser le véhicule à l'entrepôt (le dernier élément de la matrice des coûts) et explore ensuite les villes voisines. À chaque étape, le véhicule choisit le client le plus proche tout en respectant sa capacité. Si le véhicule ne peut pas desservir un autre client sans dépasser sa capacité, il retourne à l'entrepôt.

L'algorithme suit les étapes suivantes :

- Initialisation des variables : Le véhicule commence à l'entrepôt, et la tournée ainsi que le coût total sont initialisés. On crée également une liste True/False « visited » permettant de savoir si toutes nos villes ont été livrées ou non.
- Sélection du prochain client : À chaque étape, l'algorithme cherche le client le plus proche du véhicule qui respecte la capacité du véhicule et le fait qu'il n'est pas déjà été livré.
- Retour à l'entrepôt : Si un véhicule atteint sa capacité maximale, il retourne à l'entrepôt pour changer de charge et continuer la tournée.
- Finalisation de la tournée : Une fois toutes les villes desservies, le véhicule retourne au dépôt.

Une carte interactive est générée pour visualiser la tournée à l'aide la librairie geopy. Chaque ville visitée est marquée par un marqueur sur la carte, et des lignes bleues relient les villes dans l'ordre du trajet emprunté par le véhicule. La carte est sauvegardée dans un fichier HTML, permettant une visualisation facile dans un navigateur.

Le programme affiche en solution :

```
Trucks route: [7, 0, 3, 1, 7, 6, 2, 4, 7, 5, 7] (where 7 is the warehouse town)  
Total cost: 615 €  
Map of tour is save in 'map.html'.
```

- Le parcours des villes visitées par le véhicule, sous la forme d'un tableau indiquant les indices des villes dans l'ordre de la tournée.
- Le coût total de la tournée, ici 615€ calculé comme la somme des distances parcourues entre les villes successivement visitées.
- Un fichier map.html contenant la carte interactive de la tournée que voici :

